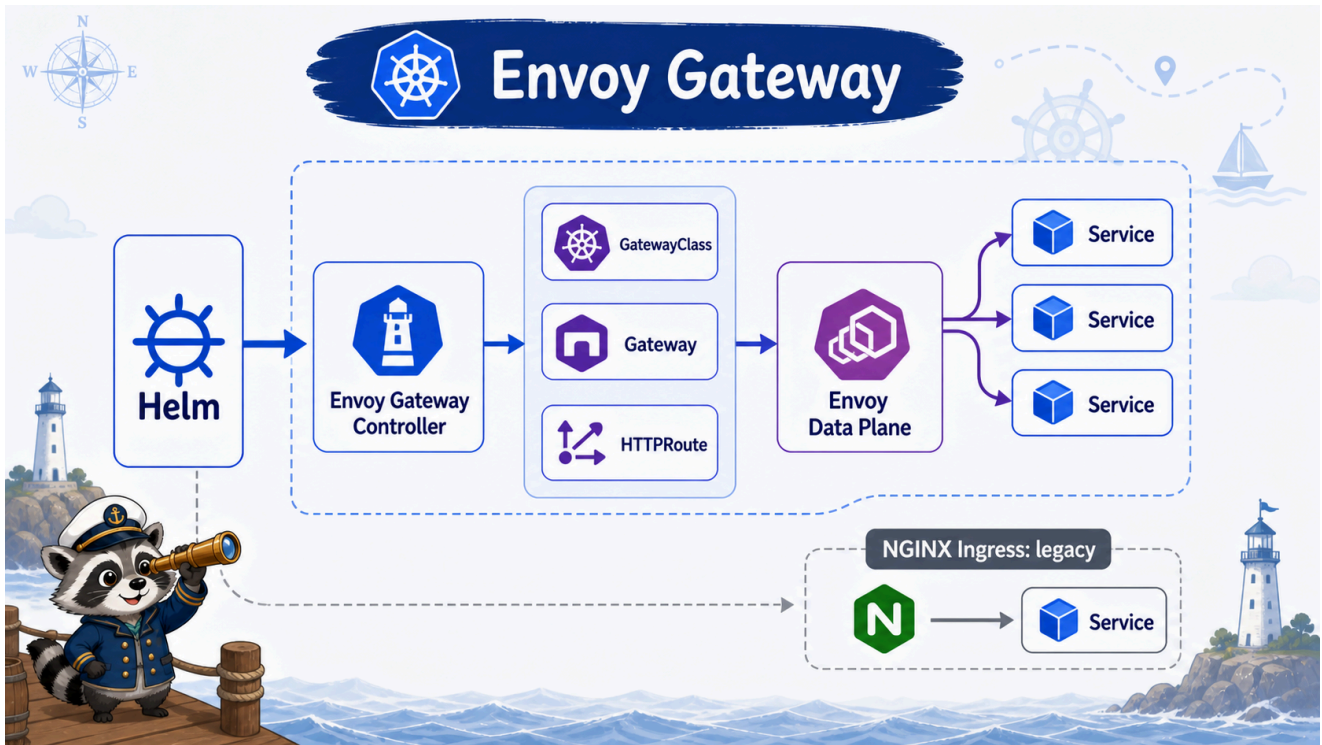


3교시: Gateway API와 Envoy Gateway 설치



수업 목표

- Gateway API의 GatewayClass, Gateway, HTTPRoute 역할을 구분한다.
- Envoy Gateway를 Helm으로 설치하고 controller Pod, GatewayClass, CRD를 확인한다.
- Docker reverse proxy 감각이 Kubernetes controller/data plane 구조로 어떻게 확장되는지 설명한다.

Docker reverse proxy에서 Kubernetes Gateway로

Docker Compose에서는 NGINX container 하나를 reverse proxy로 세우고 설정 파일에 route를 적는 방식이 흔했다.

```
browser
-> nginx reverse proxy container
-> frontend/api container
```

Kubernetes에서는 이 역할을 API object와 controller로 나눈다.

```
Gateway API object
-> GatewayClass / Gateway / HTTPRoute
```

Gateway controller

-> Envoy Gateway

data plane

-> Envoy proxy

즉 학생들이 기억해야 할 변화는 이것이다.

Docker: proxy 설정 파일을 직접 관리하는 느낌

Kubernetes: routing 의도를 API object로 선언하고 controller가 proxy를 맞춤

Gateway API 구성요소

리소스	누가 주로 관리하나	의미
GatewayClass	platform/infra team	어떤 controller가 Gateway를 처리할지 지정
Gateway	platform/app platform team	listener, port, protocol, route attach 범위
HTTPRoute	app/team owner	host/path/header 기반 routing
Service	app/team owner	backend 안정 진입점
EndpointSlice	Kubernetes controller	Ready Pod IP 목록

Ingress에서는 하나의 Ingress object에 많은 의미가 몰렸다. Gateway API는 역할을 더 잘게 나누어 platform owner와 application owner의 책임을 분리하기 쉽다.

NGINX Ingress는 왜 언급만 하는가

NGINX Ingress Controller는 오래 쓰였고 현업에서 여전히 많이 만난다. 하지만 이번 수업은 새 traffic API를 기준으로 한다.

항목	이번 수업 기준
NGINX Ingress	기존 방식으로 비교 언급
Gateway API	직접 작성
Envoy Gateway	직접 설치
HTTPRoute	직접 route 작성

현장에서 NGINX Ingress를 만나면 IngressClass, Ingress, controller log를 보면 된다. 이번 수업에서는 같은 사고방식을 GatewayClass, Gateway, HTTPRoute, Envoy Gateway log로 확장한다.

Helm values 확인

```
cat week4/day2/labs/envoy-gateway/values.yaml
```

수업 values는 최소 설정만 둔다.

```
deployment:
  replicas: 1
config:
  envoyGateway:
    logging:
      level:
        default: info
```

설치 표준은 W4 공통 원칙과 같다.

```
helm repo add/update
helm upgrade --install
namespace 명시
values file 사용
release/pod/crd 확인
```

Helm 설치

```
helm repo add eg https://gateway.envoyproxy.io
helm repo update

helm upgrade --install envoy-gateway eg/gateway-helm \
  --namespace envoy-gateway-system \
  --create-namespace \
  -f week4/day2/labs/envoy-gateway/values.yaml
```

예상 출력:

```
Release "envoy-gateway" does not exist. Installing it now.
NAME: envoy-gateway
NAMESPACE: envoy-gateway-system
STATUS: deployed
```

이미 설치되어 있으면 upgrade 출력이 나온다.

release와 controller 확인

```
helm list -n envoy-gateway-system
kubectl -n envoy-gateway-system get deploy,pod,svc
kubectl get crd | grep gateway.networking.k8s.io
```

예상:

NAME	NAMESPACE	STATUS
envoy-gateway	envoy-gateway-system	deployed
deployment.apps/envoy-gateway	1/1	
pod/envoy-gateway-...	1/1 Running	

Gateway API CRD가 있어야 GatewayClass, Gateway, HTTPRoute를 만들 수 있다.

대표 CRD:

```
gatewayclasses.gateway.networking.k8s.io
gateways.gateway.networking.k8s.io
httproutes.gateway.networking.k8s.io
```

GatewayClass 확인

```
kubectl get gatewayclass
```

예상:

NAME	CONTROLLER
envoy-gateway	gateway.envoyproxy.io/gatewayclass-controller

GatewayClass는 "이 Gateway를 어떤 구현체가 처리할 것인가"를 정한다. Ingress의 ingressClassName과 비슷한 위치에 있지만, Gateway API에서는 더 명시적인 리소스다.

controller log 읽기

```
kubectl -n envoy-gateway-system logs deploy/envoy-gateway --tail=80
```

확인할 단서:

log/상태 단서	의미
GatewayClass accepted	controller가 class를 인식
Gateway accepted	listener 설정을 처리
HTTPRoute attached	route가 Gateway에 붙음
backend not found	Service 이름/namespace 오류
invalid backend port	Service port 오류

Envoy data plane 확인

Gateway/HTTPRoute를 적용한 뒤 Envoy proxy Pod나 Service가 추가로 생길 수 있다.

```
kubectl get pods -A | grep envoy
kubectl get svc -A | grep envoy
```

환경과 chart 버전에 따라 이름이 다를 수 있다. 중요한 것은 controller Pod와 실제 traffic을 받는 data plane을 구분하는 것이다.

```
Envoy Gateway controller = API object 감시/반영
Envoy proxy data plane = 실제 HTTP traffic 처리
```

port-forward 준비

local/kind 환경에서는 LoadBalancer가 바로 외부 IP를 받지 못할 수 있다. 수업에서는 port-forward를 기본 확인 방법으로 둔다.

Gateway를 적용한 뒤 Envoy Service를 찾는다.

```
kubectl get svc -A | grep envoy
```

예시:

```
kubectl -n envoy-gateway-system port-forward svc/<envoy-service-name> 8080:80
```

다른 터미널에서 확인한다.

```
curl -H "Host: paperclip.local" http://localhost:8080/
```

아직 HTTPRoute를 만들지 않았다면 404가 나올 수 있다. 이것은 controller가 죽었다는 뜻이 아니라 route가 없다는 뜻일 수 있다.

장애 판단

증상	확인
Helm chart를 못 찾음	<code>helm repo list</code> , <code>helm repo update</code>
controller Pod Pending	<code>kubectl -n envoy-gateway-system describe pod</code>
GatewayClass 없음	<code>kubectl get gatewayclass</code>
Gateway가 Accepted 아님	<code>kubectl -n week4 describe gateway paperclip-gateway</code>
HTTPRoute가 붙지 않음	<code>kubectl -n week4 describe httproute paperclip-routes</code>
localhost 접근 실패	Envoy Service 이름, port-forward 대상 확인

Evidence Note

```
# W4D2S3 Envoy Gateway
- Helm release:
- controller Pod READY:
- GatewayClass:
- Gateway API CRD:
- Envoy Service/port-forward:
- controller log에서 본 단서:
```

한 줄 요약

Gateway API는 traffic 의도를 선언하고, Envoy Gateway는 그 의도를 Envoy data plane으로 반영하는 controller다.